AI DAY · GITHUB AGENTIC WORKFLOWS

# Autonomous Issue Triage & Implementation

From filed issue to merged PR — no human in the loop.

# Software maintenance is mostly waiting

Issues sit in backlog for days. Simple bugs that a developer could fix in 10 minutes wait weeks for someone to notice them.

Enhancements with obvious implementations get filed, labeled, and forgotten.

**The bottleneck isn't capability — it's attention.**

### A junior engineer on every issue

Reads the issue, finds the code, traces the bug, opens a PR — all within minutes of the issue being filed.

### And it never sleeps

Issues filed at 2am on a Friday get triaged and fixed before the team shows up Monday.

# Three workflows, one agentic loop

1. **Issue opened → triage-and-implement**
   Classifies as bug / enhancement / question / needs-more-info. For bugs and enhancements: reads source, investigates, opens a `fix:` or `feat:` PR with the correct label if confident. Always posts a triage summary.

2. **`/fix <description>` on a PR → pr-review**
   Reads the PR diff and comment thread, applies the requested change, pushes a new commit, replies with a summary. Triggered only on PR comments — not issues.

3. **PR opened → pr-risk-review**
   Skeptical second-opinion review. Reads the full diff and surrounding file context, flags logic errors, edge cases, regressions, and API contract changes. Submits a `REQUEST_CHANGES` review on high severity, `COMMENT` otherwise.

All three run on **Claude Sonnet** via the `engine: claude` field. The target codebase is a small Next.js todo app.

# GitHub Agentic Workflows: the format

A `.md` file with YAML front matter + a natural language prompt. Compiled to standard Actions YAML via `gh aw compile`.

The front matter declares **what** the agent is allowed to do. The prompt describes **how** it should think.

This separation matters: constraints live in config, not in prose that the model might reason around.

```
on:
  issues:
    types: [opened]
engine: claude
permissions:
  contents: read
  issues: read
safe-outputs:
  create-pull-request:
  add-labels:
  add-comment:
tools:
  github:
    toolsets: [default]
---

# Your prompt here…
```

## Safe Outputs

The agent cannot write to GitHub directly. All write operations are declared upfront, batched, and executed by the framework **after** the agent finishes — with full auditability.

### Declared intent

You list every write action the agent is permitted. It cannot do anything not on the list — no surprise side effects.

### Atomic execution

Outputs are applied together at the end. If a push fails, remaining outputs are cancelled — no partial state.

### Full audit trail

The Actions log shows every tool call, every API request, and every output the agent produced — nothing is hidden.

# What the audit trail looks like

Every workflow run in Actions shows a structured log of the agent's work — what it read, what it decided, what it wrote:

```
▶ Reading issue #7: "Item count is always one less than expected"
▶ Searching repository for countRemaining...
GET /repos/dcow/ai-day-agentic-triage-fix/contents/lib/todos.ts
▶ Found off-by-one: loop starts at i=1, skips first todo
▶ Writing fix...
POST /repos/.../pulls → PR #8 created
▶ Safe outputs executed:
  ✓ create-pull-request: fix/count-remaining-off-by-one
  ✓ add-labels: bug
  ✓ add-comment: triage summary posted
```

The firewall layer also surfaces any unexpected network calls — you can see exactly what APIs the agent touched.

# Slash commands: /fix on PRs

The `slash_command` trigger type filters at the **action predicate level** — the agent is never invoked unless the exact command matches.

No polling, no agent spin-up for unrelated comments. The activation condition is compiled directly into the Actions YAML `if:` expression.

```
on:
  slash_command:
    name: fix
    events: [pull_request_comment]
```

Using `pull_request_comment` (not `issue_comment`) scopes it to PRs only at the predicate level.

### Usage

Comment on any PR:

```
/fix rename the button label
    from "Add" to "Add todo"
```

### What happens

- Agent reads PR diff + comment thread
- Applies the change surgically
- Pushes a new commit to the branch
- Replies: **Done —** ...

# Automated risk review: a skeptical second opinion

Every PR — whether opened by a human or by the triage agent itself — gets reviewed by a second Claude instance acting as a skeptical senior engineer.

It reads the full diff **and** the surrounding context of changed files. Not just "what changed" but "what could this break."

### XSS in the wild

We opened a PR swapping `{todo.text}` for `dangerouslySetInnerHTML`. The reviewer flagged it as **high severity** and submitted `REQUEST_CHANGES` — blocking the merge.

```
safe-outputs:
  submit-pull-request-review:

# Reviews for:
# - Logic errors & off-by-one
# - Edge cases (null, empty, concurrent)
# - Regressions (callers, invariants)
# - Error handling gaps
# - API/contract changes
#
# high   → REQUEST_CHANGES
# medium → COMMENT
# low    → COMMENT
# none   → "No significant risks
#           identified."
```

Critically: it's told to say nothing if nothing is wrong. No invented risks to appear thorough.

# Key insights

### Safe outputs = the right abstraction

Declaring write permissions upfront in config — not prose — means the model can't reason its way into doing something it wasn't allowed. Constraints enforced by the framework, not the prompt.

### Predicate filtering > prompt filtering

Use `slash_command` triggers and the right event types to gate agent invocation at the Actions level. Prompt-level bail is a last resort — by then you've already paid the cost.

### Reactions solve "dead silence"

Safe outputs batch at the end — no mid-run feedback. The `reaction:` field fires immediately at activation (👀 on issues, 🚀 on `/fix`) so users know the agent is working.

### Two Claudes > one Claude

The triage agent opens a PR; the risk reviewer catches what it missed. Separate agents with separate prompts and separate trust levels are more robust than one agent doing everything.

### Don't fight the batching

All safe outputs execute at the end. Accept it and design for it — reactions cover the UX gap without splitting workflows unnecessarily.

### Implementation, not just triage

The same agent that classifies issues writes production code and opens PRs. Don't undersell the engine — give it implementation tasks.

# The issue tracker becomes a product spec

When the gap between "file an issue" and "running in production" closes to minutes, the way teams work changes. You stop triaging backlogs and start writing better issues.

The constraint shifts from **developer time** to **issue quality**. Well-specified = automatically implemented. Vague = asked for more detail.

- Issues → PRs automatically
- PR comments → new commits
- Risk review on every PR
- No agent hallucinations into prod